

Developer Contribution Visualizer

1. Problem understanding

- **Input:** Git repository URLs, API authentication tokens, and target date range parameters.
- **Processing:** The system fetches repository commit history, parses lines of code (LOC) added and deleted, categorizes the actions by author, and calculates frequency, volume, and impact metrics.
- **Output:** Visual dashboards featuring graphs and heatmaps that display individual and team contribution metrics over time.

2. Actors/Stakeholders

- **Engineering Managers:**
 - *Contribution:* Input repository access credentials and set tracking parameters.
 - *Expectation:* Clear visibility into team velocity, bottleneck identification, and objective performance data.
- **Developers/Contributors:**
 - *Contribution:* Write code and push commits to the repository.
 - *Expectation:* Accurate representation of their work without arbitrary gamification that penalizes complex, low-LOC debugging or refactoring.
- **Project Managers:**
 - *Contribution:* Align the visualizer's output with active sprint goals.
 - *Expectation:* Predictable data to track if technical execution aligns with scheduled timelines.

3. Associated Applications

- **Version Control Systems (e.g., GitHub, GitLab):** Act as the primary data sources. The visualizer associates with them via API calls to extract commit logs, pull request statuses, and issue resolutions.
- **Project Management Tools (e.g., Jira):** Used to cross-reference code contributions with active workflows. The visualizer can parse commit messages for issue keys to map code output directly back to specific backlog epics and user stories.

4. Functional and Non-functional Properties

Functional:

- The system must authenticate users via OAuth.
- The system must fetch, parse, and aggregate commit data for specified date ranges.
- The system must render dynamic visual charts (e.g., line graphs for frequency, heatmaps for activity density).
- **Non-functional:**

- *Performance:* Fetching and rendering data for a repository with up to 10,000 commits must complete in under 5 seconds.
- *Security:* The system must only process and store metadata; it must never clone or store the actual source code locally.
- *Reliability:* The parsing engine must gracefully handle and log external API rate limit errors without crashing the frontend.

5. System Components and Interaction

- **Frontend Interface:** Captures user inputs (repository links, filter settings) and renders the visual dashboards.
- **API Gateway:** Receives requests from the frontend and routes them appropriately.
- **Data Parser:** The core backend engine that connects to external Git APIs, extracts the raw JSON commit data, and calculates the required metrics.
- **Database:** Caches the processed metadata to reduce redundant API calls and prevent rate limiting.
- *Interaction Flow:* The frontend requests data -> the API Gateway routes to the Data Parser -> the Data Parser checks the Database for cached data -> if none exists, it fetches from the Git API, processes the metrics, stores the result in the Database, and returns the JSON payload to the frontend for rendering.

6. Technology Stack

- **Frontend:** React.js or Vue.js paired with a specialized visualization library like D3.js or Chart.js.
- **Backend:** Node.js (Express) or Python (FastAPI) for handling API requests and parsing logic.
- **Database:** PostgreSQL for structured relational data (user profiles, repositories, cached commit statistics).
- **Infrastructure:** RESTful APIs or GraphQL for interacting with the Git providers.

7. Assumptions and Modifications

- *Assumption 1:* Developers use consistent email addresses across their commits. If a developer uses multiple emails, the system will incorrectly split their contributions unless manual profile merging is implemented.
- *Assumption 2:* Target repositories utilize standard Git version control conventions.
- *Assumption 3:* External Git provider API rate limits are sufficient for the intended volume of data requests under standard usage conditions.